Introduction to Programming

Course Information

Administrative details

- **Evaluation**
  - In-class quizzes 20%
  - Assignments 20%
  - Midsemester exam 25%
  - Final exam 35%
- **Teaching assistants**
  - Ryan Hota (MSc 2): `ryan@cmi.ac.in`
  - Soham Saha (BSc 2): `sohams.ug2025@cmi.ac.in`
  - Tejas Sah (BSc 2): `tejass.ug2025@cmi.ac.in`
- **Primary references**
  - *Learn You a Haskell for Great Good!* (read online)
    `https://learnyouahaskell.github.io`
  - *Programming in Haskell*, Graham Hutton (Cambridge University Press)
  - *Thinking Functionally with Haskell*, Richard Bird (Cambridge University Press)
  - *Haskell for CMI*, Arjun Maneesh Agarwal, Ryan Hota and Shubh Sharma
    `https://haskell-for-cmi.vercel.app/release.pdf`
  - Introduction To Haskell Programming (NPTEL course)
    `https://nptel.ac.in/courses/106106931`
- **More references**
  - Official Haskell page
    `https://www.haskell.org`
  - *Introduction to Functional Programming using Haskell*, Richard Bird (Prentice Hall)
  - Haskell Libraries: Documentation
    `https://downloads.haskell.org/~ghc/latest/docs/html/libraries/`
  - Haskell for Imperative Programmers: YouTube playlist
    `https://www.youtube.com/playlist?list=PLe7Ei6viL6jGp1Rfu0dil1JH1SHk9bgDV`
- Submit all assignments on *this Moodle page only*. Further instructions will be given as part of the assignments.

Course plan

- Introduction to Haskell and the `ghci` interpreter
- Defining functions: guards, pattern matching and recursion
- Lists, recursive functions on lists
- Basic list functions and list comprehension
- Higher-order functions on lists: `map`, `filter`, `foldr`, `foldl`, etc.
- Defining new data types, data and type constructors
- Polymorphism, type classes
- Computation as rewriting, lazy evaluation and infinite lists
- Input/output and standalone programs with a `main`
- IO: repeatedly performing tasks, looping
- Measuring efficiency of programs

- Sorting algorithms
- Breadth-first search and depth-first search, examples
- Modules and abstract data types: stacks, queues, etc.
- Data structures: binary search trees, AVL trees, heaps, etc.
- Memoization using arrays
- Introduction to monads through examples

Installing Haskell

1. We will be using GHC, also known as the Glasgow Haskell Compiler.
2. Instructions to install Haskell can be found at `https://www.haskell.org/downloads/`.
3. Please contact the TAs if you want further help, and if you need a specific set-up to follow the *Haskell for CMI* book.

Writing and running programs

I shall be using Visual Studio Code on macOS to write my programs, and shall be using `ghci` on the Terminal to run programs. GHCi (GHC interactive) is a very nice REPL that will be indispensable for us in learning Haskell. Later I shall also compile programs using `ghc` to produce executable programs and run them on the terminal. So you should be prepared to do a lot of work on the terminal. It is a good thing to learn in general.

You should learn to use a decent editor, which will go a long way towards making your computer use productive for the rest of your life. It is recommended that you get comfortable with either *vi* or *emacs*. Treat that as an essential part of your CS education, and learn it in your spare time.

Finally

Please feel free to contact me or the TAs anytime you need help with anything.

Midsemester exam syllabus

Added to this copy; this section is not on the Moodle course page.

The material for the midsemester exam is everything in the slides covered so far, up to slide 14 of the slide deck titled *More folds*. In the *Lecture Slides* folder this is:

- the decks *01-functions-and-types* to *08-higher-order*, in full;
- *09-more-folds*, slides 1 to 14 of 24: `foldr`, `foldr1` and `foldl`.

So the following are not part of the syllabus: `foldl'`, `seq`, `$!`, `scanr` and `scanl`, simulating `foldl` with `foldr` and `foldr` with `foldl`, and everything after these in the deck and in later decks.